OBJECT ORIENTED SOFTWARE ENGINEERING

Ms. Archana A

Department of Computer Applications

OBJECT ORIENTED SOFTWARE ENGINEERING

Design Concepts…

Archana A
Department of Computer Applications

The Design Concepts

Design concepts provide the **necessary framework** "**to get the thing on right way**".

1. Abstraction.
2. Architecture.
3. Patterns.
4. Separation of concerns.
5. Modularity
6. Information Hiding.
7. **Functional independence.**
8. **Refinement**
9. **Aspects**
10. Refactoring.
11. Object Oriented design concepts
12. Design Classes.

7. Functional Independence

- The concept of functional independence is a **direct outgrowth** of **separation of concerns, modularity, and the concepts of abstraction and information hiding.**

- Functional independence is achieved by **developing modules** with "**single-minded**" function and an "**aversion**“ (dislike) to excessive interaction with other modules.

7. Functional Independence…

Why independence is important?

- Independent modules are **easier to maintain** (and test) because
  - **code modification are limited**,
  - **error propagation is reduced, and**
  - **reusable modules** are possible.

- **Functional independence** is a **key** to **good design**, and good design is the key to software quality.

- Independence is assessed using **two qualitative criteria:**
  - ***Cohesion*** **and** ***Coupling.***

7. Functional Independence…

- ***Cohesion*** is an indication of the relative **functional strength of a module.** (functional relatedness of elements within a module).
  - A cohesive module performs a **single task,** requiring **little interaction** with other components in other parts of a program.
  - Stated simply, a cohesive module should (ideally) **do just one thing.** Which means no much dependency among modules i.e. **High Cohesion**.

7. Functional Independence…

- ***Coupling*** is an indication of the relative **interdependence among modules**.
  - Coupling depends on the **interface complexity between modules**, the point at which entry or reference is made to a module, and what data pass across the interface.
  - It is always good if there is **Low coupling**.

8. Refinement

- Refinement is actually a **process of elaboration.**

- This begin with a **statement of function** (or description of information) that is **defined** at a **high level of abstraction.**

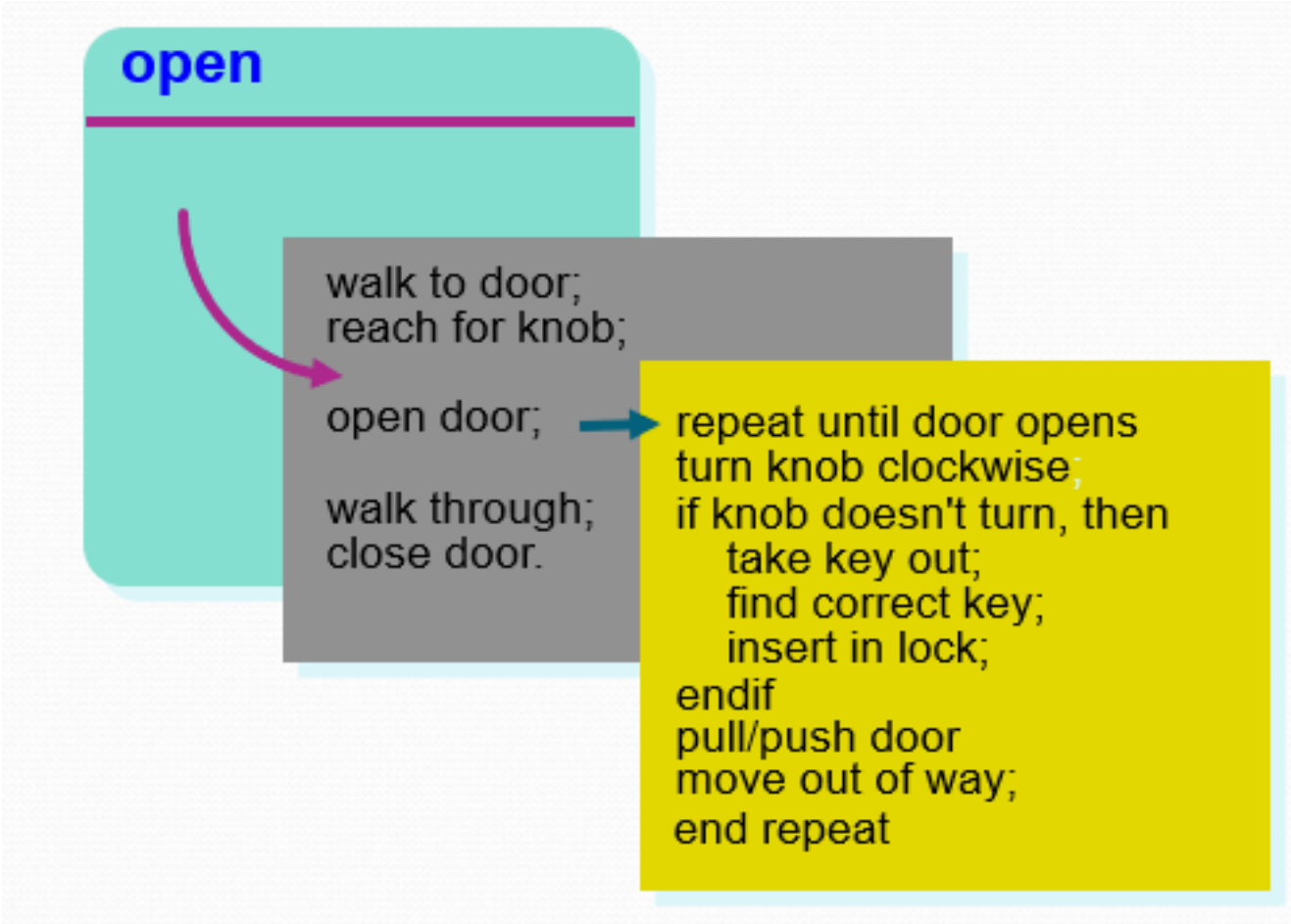
8. Refinement…

- The **statement describes** function or information **conceptually** but provides **no information about the internal workings** of the function or the **internal structure** of the information.

- **Refinement** causes the designer to **elaborate on the original statement**, providing **more and more detail** as each **successive refinement** (elaboration) occurs.

8. Refinement…

- Stepwise Refinement



8. Refinement…

- Abstraction and Refinement are **complementary concepts.**
- **Abstraction** enables a designer to specify procedure and data and yet **suppress** **low-level details.**
- **Refinement** helps the designer to **expose** **low-level details** as design progresses.

9. Aspects

In software design, **"aspects"** refer to a **specific approach** for modularizing **cross-cutting concerns** that are **not** naturally encapsulated **within individual objects or modules.**

What are Cross-cutting concerns ?

- These are functionalities that affect multiple parts of the system, such as **logging, security, caching**, or **transaction management** etc.

By using aspects, you can **separate these concerns** into **independent modules, promoting better code organization, maintainability, and reusability.**

Examples:

Logging: Imagine an e-commerce application.

Logging user actions is a cross-cutting concern affecting various modules (**authentication, shopping cart, checkout**). Instead of scattering logging code throughout, an aspect could intercept **specific method calls** (pointcuts) and log relevant information (advice). This makes logging code more modular and easier to maintain.

Examples:

Security: In a banking application, security checks are crucial.

Instead of repeating security checks in every module, an aspect could intercept **method calls** related to sensitive data (pointcuts) and perform authorization checks or encryption (advice). This ensures consistent security across the application.

Benefits of using aspects:

1. **Improved modularity:** Concerns are separated into independent units, making code easier to understand and maintain.
2. **Reduced code duplication:** Cross-cutting logic is written once and reused across different parts of the system.
3. **Enhanced flexibility:** New aspects can be added or modified without affecting core functionality.

THANK YOU

Ms. Archana A
Department of Computer Applications

archana@pes.edu

+91 80 6666 3333 Extn 392